

Receiver-Guided Reinforcement Learning-based Congestion Control for IoT

Qihang Jiao, *Student Member, IEEE*, and Lotfi Mhamdi, *Senior Member, IEEE*

Abstract—The abstract goes here.

Index Terms—IoT, TCP, congestion control, reinforcement learning.

I. INTRODUCTION

THIS part is to be completed.

II. RELATED WORKS

III. SYSTEM DESIGN

Our proposal contains three modules in the receiver side for adjusting sender *cwnd*, Receiver Measurement Module, Window Decision Module, and Window Enforcement Module. Additionally, TCP sockets in both sender and receiver are extended or modified for adding interfaces to achieved our features.

A. Message types definition

The receiver TCP socket is extended to support communication with the RL agent. Three types of messages are defined:

- **State**: carries the observed network state and is sent to the agent for storage in the replay buffer.
- **Action Request**: triggers the agent to infer and return a control action based on the current state.
- **Connection Close**: notifies the agent that a flow has terminated, allowing it to clear the corresponding internal states.

B. Receiver Measurement Module

Our scheme relies on reliable state observations for a stable training. For a receiver, RTT is not able to be directly observed while it is valuable metric to measure the congestion extent of a network. We overcome this by extending the sender TCP socket to include their *cwnd* in TCP header. In standard TCP congestion avoidance, CC increases increase by approximately 1 MSS after a RTT, i.e., when an entire *cwnd* bytes are acknowledged. We utilise this to enable the receiver to estimate network congestion extent by calculating the elapsed time for a flow to complete its *cwnd*. Therefore, upon every packet receive event that includes a *cwnd* value different from the last value received, the Receiver Measurement Modules establishes a new estimation task. A new task is initialised with the current

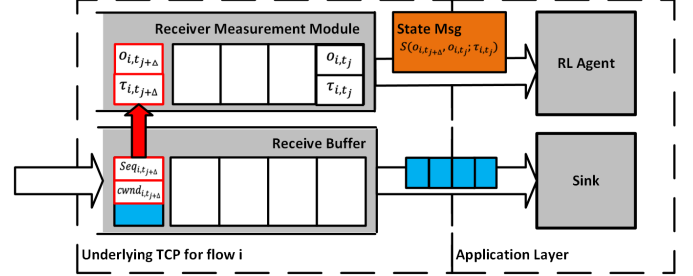


Fig. 1. An example of estimation task completion of flow *i*.

observations and a threshold, e.g., for flow *i* at time *j*, defined as below:

$$\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j}$$

An estimation task is completed when a coming packet is seen with $Seq_{i,t_j+\Delta} > \tau_{i,t_j}$. The *cwnd* elapsed time is calculated as $t_{elapsed} = t_{j+\Delta} - t_j$. Then $t_{elapsed}$ along with other observations are processed to generate a state for training our RL Agent. In observations, we maintain a cumulative received bytes, packet losses i.e., an observation of flow *i* at *t_j* defined as follow:

$$o_{i,t_j} \triangleq [B_i(t_j), L_i(t_j), t_j] \quad (1)$$

$$B_i(t_j) = \sum_{n=0}^{t_j} RxBytes_{i,n}$$

$$L_i(t_j) = \sum_{n=0}^{t_j} Losses_{i,n}$$

Then the goodput (g_{i,t_j}) and the packet loss rate (ℓ_{i,t_j}) are calculated and formed a state message along with the $t_{elapsed}$. The state generation is defined as:

$$S(o_{i,t_j+\Delta}, o_{i,t_j}; \tau_{i,t_j}) = [g_{i,t_j}, \ell_{i,t_j}, t_{elapsed}]$$

where $g_{i,t_j} = \frac{B_i(t_{j+\Delta}) - B_i(t_j)}{t_{elapsed}}$ and $\ell_{i,t_j} = \frac{L_i(t_{j+\Delta}) - L_i(t_j)}{t_{elapsed}}$.

C. Window Decision Module

The states input is triggered by each receive event that is not able to be known when it happens. Also for the purpose to avoid the agent decision process blocking the normal TCP behaviour, we set a window decision maintained in the Receiver socket and updated periodically. The window decision is a value within $[0, 1]$ and set in every ACK header to the sender.

Qihang Jiao and Lotfi Mhamdi are with the School of Electronic and Electrical Engineering, University of Leeds, Leeds, UK (e-mail: el22qj@leeds.ac.uk; L.Mhamdi@leeds.ac.uk).

Lotfi Mhamdi is also with the College of Computing & Systems, Abdullah Al Salem University, Kuwait (e-mail: lotfi.mhamdi@asu.edu.kw).

Algorithm 1 Receiver Measurement and State Construction

Input: Flow Id i , Time t_j , Arrived Packet p_{i,t_j} , Flow Observation Queue q_i

Output: State message s_{i,t_j}

- 1: Extract Seq_{i,t_j} , $cwnd_{i,t_j}$, and $RxBytes_{i,t_j}$ from p_{i,t_j}
 - 2: $\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j}$
 - 3: Obtain $Losses_{i,t_j}$ from $TcpSocket_i$
 - 4: $B_i(t_j) = B_i(t_{j-1}) + RxBytes_{i,t_j}$
 - 5: $L_i(t_j) = L_i(t_{j-1}) + Losses_{i,t_j}$
 - 6: $o_{i,t_j} = [B_i(t_j), L_i(t_j), t_j]$
 - 7: Enqueue o_{i,t_j} and τ_{i,t_j} into q_i
 - 8: Obtain o_i^{head} and τ_i^{head} from the head of q_i
 - 9: **if** $Seq_{i,t_j} > \tau_i^{head}$ **then**
 - 10: Dequeue o_i^{head} and τ_i^{head} from q_i
 - 11: $s_{i,t_j} = S(o_{i,t_j}, o_i^{head})$
 - 12: Send s_{i,t_j} as State Message to Agent
 - 13: **end if**
-

D. Window Enforcement Module

The sender is modified to leverage the window decision as a factor to update its $cwnd$ as following:

$$cwnd \leftarrow cwnd \times Scaling_Factor^{Decision}$$

E. Enforcing Window Generation by RL Agent

IV. SIMULATION SETUP AND RESULTS

V. CONCLUSION

The conclusion goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

- [1] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.